//1//
Quick Sort is a divide-and-conquer sorting algorithm

Choose a pivot element from the array.
Partition the array so that:
Elements smaller than the pivot are placed on the left side.
Elements greater than the pivot are placed on the right side.
Recursively apply Quick Sort to the left and right subarrays.

Time complexity : O(n log n)

```
#include <stdio.h>

int comparisons = 0;
int recursiveCalls = 0;

void swap(int *a, int *b)
{
    int temp = *a;
    *a = *b;
    *b = temp;
}

int partition(int arr[], int low, int high)
{
    int pivot = arr[high];
    int i = low - 1;

    for(int j = low; j < high; j++)
    {
        comparisons++;
        if(arr[j] < pivot)
        {
            i++;
            swap(&arr[i], &arr[j]);
        }
    }

    swap(&arr[i + 1], &arr[high]);
    return i + 1;
}

void quickSort(int arr[], int low, int high)
{
    if(low < high)
    {
        recursiveCalls++;

        int pi = partition(arr, low, high);

        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}

int main()
{
    int n, arr[50];

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter elements:\n");
    for(int i = 0; i < n; i++)
        scanf("%d", &arr[i]);
```

```
    quickSort(arr, 0, n - 1);

    printf("\nSorted array:\n");
    for(int i = 0; i < n; i++)
        printf("%d ", arr[i]);

    printf("\n\nComparisons = %d", comparisons);
    printf("\nRecursive Calls = %d\n", recursiveCalls);

    return 0;
}
```

//2//

The Divide and Conquer technique solves a problem by dividing it into smaller subproblems, solving them recursively, and combining the results.
For computing large positive integer power, the traditional approach multiplies the base repeatedly:

Time complexity: O(log n)

```
#include <stdio.h>

int multiplications = 0;

long long power(long long a, int n)
{
    if(n == 0)
        return 1;

    long long half = power(a, n / 2);

    if(n % 2 == 0)
    {
        multiplications++;
        return half * half;
    }
    else
    {
        multiplications += 2;
        return a * half * half;
    }
}

int main()
{
    long long a;
    int n;

    printf("Enter base: ");
    scanf("%lld", &a);

    printf("Enter exponent: ");
    scanf("%d", &n);

    long long result = power(a, n);

    printf("\nResult = %lld", result);
    printf("\nMultiplications = %d\n", multiplications);

    return 0;
}
```

//3//

Merge Sort repeatedly divides the array until each subarray contains only one element. A single-element array is already sorted.
Time complexity: O(nlogn)

```
#include <stdio.h>

int comparisons = 0;

void merge(int arr[], int l, int m, int r)
{
    int i, j, k;
    int n1 = m - l + 1;
    int n2 = r - m;

    int L[50], R[50];

    for(i = 0; i < n1; i++)
        L[i] = arr[l + i];

    for(j = 0; j < n2; j++)
        R[j] = arr[m + 1 + j];

    i = 0;
    j = 0;
    k = l;

    while(i < n1 && j < n2)
    {
        comparisons++;
        if(L[i] <= R[j])
            arr[k++] = L[i++];
        else
            arr[k++] = R[j++];
    }

    while(i < n1)
        arr[k++] = L[i++];

    while(j < n2)
        arr[k++] = R[j++];
}

void mergeSort(int arr[], int l, int r)
{
    if(l < r)
    {
        int m = (l + r) / 2;

        mergeSort(arr, l, m);
        mergeSort(arr, m + 1, r);
        merge(arr, l, m, r);
    }
}

int main()
{
    int n, arr[50];

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter elements:\n");
    for(int i = 0; i < n; i++)
        scanf("%d", &arr[i]);
```

```
    mergeSort(arr, 0, n - 1);

    printf("\nSorted array:\n");
    for(int i = 0; i < n; i++)
        printf("%d ", arr[i]);

    printf("\nComparisons = %d\n", comparisons);

    return 0;
}

//4//
The Fractional Knapsack problem is an optimization problem where we maximize
profit while staying within the knapsack capacity.
Time complexity O(nlogn)
#include <stdio.h>

struct Item
{
    int weight, profit;
    float ratio;
};

void sort(struct Item items[], int n)
{
    for(int i = 0; i < n - 1; i++)
    {
        for(int j = i + 1; j < n; j++)
        {
            if(items[i].ratio < items[j].ratio)
            {
                struct Item temp = items[i];
                items[i] = items[j];
                items[j] = temp;
            }
        }
    }
}

int main()
{
    struct Item items[20];
    int n, capacity;

    printf("Enter number of items: ");
    scanf("%d", &n);

    for(int i = 0; i < n; i++)
    {
        printf("Profit and Weight of item %d: ", i + 1);
        scanf("%d %d", &items[i].profit, &items[i].weight);
        items[i].ratio = (float)items[i].profit / items[i].weight;
    }

    printf("Enter knapsack capacity: ");
    scanf("%d", &capacity);

    sort(items, n);

    float totalProfit = 0.0;

    for(int i = 0; i < n; i++)
    {
```

```
        if(capacity >= items[i].weight)
        {
            capacity -= items[i].weight;
            totalProfit += items[i].profit;
        }
        else
        {
            totalProfit += items[i].ratio * capacity;
            break;
        }
    }

    printf("Maximum Profit = %.2f\n", totalProfit);

    return 0;
}
```

//5//
A Minimum Spanning Tree (MST) of a connected, undirected weighted graph is a subset of edges that:

Connects all vertices
Has no cycles
Has minimum possible total edge weight

Greedy algorithms are commonly used to find MST.

```
//prim's
#include <stdio.h>
#define INF 999

int main()
{
    int n, cost[20][20], visited[20] = {0};
    int min, mincost = 0;
    int a, b, u, v;

    printf("Enter number of vertices: ");
    scanf("%d", &n);

    printf("Enter cost matrix:\n");
    for(int i = 0; i < n; i++)
    {
        for(int j = 0; j < n; j++)
        {
            scanf("%d", &cost[i][j]);
            if(cost[i][j] == 0)
                cost[i][j] = INF;
        }
    }

    visited[0] = 1;

    printf("\nEdges in MST (Prim):\n");

    for(int ne = 1; ne < n; ne++)
    {
        min = INF;

        for(int i = 0; i < n; i++)
        {
            for(int j = 0; j < n; j++)
            {
                if(cost[i][j] < min)
```

```
                    if(visited[i] != 0)
                    {
                        min = cost[i][j];
                        a = u = i;
                        b = v = j;
                    }
                }
            }

            if(visited[u] == 0 || visited[v] == 0)
            {
                printf("%d - %d : %d\n", a, b, min);
                mincost += min;
                visited[b] = 1;
            }

            cost[a][b] = cost[b][a] = INF;
        }

        printf("Minimum Cost = %d\n", mincost);
        return 0;
    }

    //Kruskals
    #include <stdio.h>
    #define MAX 20

    int parent[MAX];

    int find(int i)
    {
        while(parent[i])
            i = parent[i];
        return i;
    }

    int uni(int i, int j)
    {
        if(i != j)
        {
            parent[j] = i;
            return 1;
        }
        return 0;
    }

    int main()
    {
        int n, cost[MAX][MAX];
        int mincost = 0, ne = 1;
        int a, b, u, v, min;

        printf("Enter number of vertices: ");
        scanf("%d", &n);

        printf("Enter cost matrix:\n");
        for(int i = 0; i < n; i++)
            for(int j = 0; j < n; j++)
            {
                scanf("%d", &cost[i][j]);
                if(cost[i][j] == 0)
                    cost[i][j] = 999;
            }
```

```
    printf("\nEdges in MST (Kruskal):\n");

    while(ne < n)
    {
        min = 999;

        for(int i = 0; i < n; i++)
            for(int j = 0; j < n; j++)
                if(cost[i][j] < min)
                {
                    min = cost[i][j];
                    a = u = i;
                    b = v = j;
                }

        u = find(u);
        v = find(v);

        if(uni(u, v))
        {
            printf("%d - %d : %d\n", a, b, min);
            mincost += min;
            ne++;
        }

        cost[a][b] = cost[b][a] = 999;
    }

    printf("Minimum Cost = %d\n", mincost);
    return 0;
}
```

//6//
The Traveling Salesman Problem (TSP) is a classic optimization problem in which a salesman must visit each city exactly once and return to the starting city while minimizing the total travel cost.

```
#include <stdio.h>
#include <limits.h>

#define MAX 15
#define INF 999999

int n;
int cost[MAX][MAX];
int dp[1 << MAX][MAX];

int min(int a, int b)
{
    return (a < b) ? a : b;
}

int tsp(int mask, int pos)
{
    if(mask == (1 << n) - 1)
        return cost[pos][0]; // return to start

    if(dp[mask][pos] != -1)
        return dp[mask][pos];

    int ans = INF;

    for(int city = 0; city < n; city++)
    {
```

```
        if((mask & (1 << city)) == 0)
        {
            int newAns = cost[pos][city] +
                         tsp(mask | (1 << city), city);
            ans = min(ans, newAns);
        }
    }

    return dp[mask][pos] = ans;
}

int main()
{
    printf("Enter number of cities: ");
    scanf("%d", &n);

    printf("Enter cost matrix:\n");
    for(int i = 0; i < n; i++)
        for(int j = 0; j < n; j++)
            scanf("%d", &cost[i][j]);

    // initialize dp with -1
    for(int i = 0; i < (1 << n); i++)
        for(int j = 0; j < n; j++)
            dp[i][j] = -1;

    int result = tsp(1, 0);

    printf("\nMinimum Traveling Cost = %d\n", result);

    return 0;
}

//7//
Backtracking is a problem-solving technique that builds solutions step by step
and removes those solutions that fail to satisfy the given constraints

#include <stdio.h>
#include <stdlib.h>

#define MAX 20

int x[MAX]; // stores column positions
int n;

int place(int k, int i)
{
    for(int j = 1; j < k; j++)
    {
        if(x[j] == i || abs(x[j] - i) == abs(j - k))
            return 0;
    }
    return 1;
}

void nQueens(int k)
{
    for(int i = 1; i <= n; i++)
    {
        if(place(k, i))
        {
            x[k] = i;

            // show traversal step
```

```
            printf("\nPlacing queen at row %d column %d", k, i);

            if(k == n)
            {
                printf("\nSolution: ");
                for(int j = 1; j <= n; j++)
                    printf("%d ", x[j]);
            }
            else
            {
                nQueens(k + 1);
            }
        }
        else
        {
            // show backtracking
            printf("\nBacktracking from row %d column %d", k, i);
        }
    }
}

int main()
{
    printf("Enter number of queens: ");
    scanf("%d", &n);

    nQueens(1);

    return 0;
}

//8//
The 0/1 Knapsack problem is a classic optimization problem where we must select
items to maximize total profit without exceeding the knapsack capacity.
time complexity O(nw)

#include <stdio.h>

int max(int a, int b)
{
    return (a > b) ? a : b;
}

int main()
{
    int n, W;
    int weight[20], profit[20];
    int dp[20][50];

    printf("Enter number of items: ");
    scanf("%d", &n);

    printf("Enter weights:\n");
    for(int i = 1; i <= n; i++)
        scanf("%d", &weight[i]);

    printf("Enter profits:\n");
    for(int i = 1; i <= n; i++)
        scanf("%d", &profit[i]);

    printf("Enter knapsack capacity: ");
    scanf("%d", &W);

    for(int i = 0; i <= n; i++)
```

```
    {
        for(int w = 0; w <= W; w++)
        {
            if(i == 0 || w == 0)
                dp[i][w] = 0;
            else if(weight[i] <= w)
                dp[i][w] = max(dp[i-1][w],
                               profit[i] + dp[i-1][w-weight[i]]);
            else
                dp[i][w] = dp[i-1][w];
        }
    }

    printf("\nMaximum Profit = %d\n", dp[n][W]);

    return 0;
}
```